

Manual of DIMFormer-GAN

Repository: <https://github.com/shengdate/DIMFormer-GAN>

1 Introduction

DIMFormer-GAN is a Python/PyTorch framework developed for automatic extraction and separation of multimodal surface-wave dispersion curves from dispersion-energy spectra. The method treats the target dispersion energy as a single semantic foreground class while separating different propagation modes as independent instances. The released repository provides the core network implementation, the training and inference scripts, a pretrained model, a sample dataset, and example test images.

The repository is organized around two basic workflows:

1. Model training using the sample VOC-style dataset and train.py.
2. Pretrained-model inference using test.py, the checkpoint best_model.pth, and the example images stored in the test directory.

For routine use, the recommended first test is the pretrained-model inference workflow because it can be executed directly without retraining the network.

2 Prerequisites

DIMFormer-GAN was prepared for the following Python/PyTorch environment. The package versions below should be used when reproducing the released training and inference workflows:

Package	Version
Python	3.9
PyTorch	2.0.0+cu118
torchvision	0.15.1+cu118
NumPy	1.24.3
Pillow	11.3.0
Matplotlib	3.9.2
opencv-python	4.11.0.86
SciPy	1.13.1
pandas	2.3.3

A separate Conda environment is recommended to avoid conflicts between Python packages. A typical setup is:

```
conda create -n dimformer python=3.9
conda activate dimformer
pip install torch==2.0.0+cu118 torchvision==0.15.1+cu118 --index-url
https://download.pytorch.org/whl/cu118
pip install numpy==1.24.3 Pillow==11.3.0 matplotlib==3.9.2 opencv-python==4.11.0.86 scipy==1.13.1
pandas==2.3.3
```

The +cu118 PyTorch and torchvision builds are used with the CUDA 11.8 runtime. GPU acceleration is recommended, although the released inference script automatically selects the CPU when CUDA is not available.

DIMFormer-GAN can be run directly from the repository and does not require installation as a Python package.

3 Repository organization and input data

The expected repository structure is:

```
DIMFormer-GAN/
|-- README.md
|-- requirements.txt
```

```
| -- doc/  
|   |-- manual_DIMFormer-GAN_editable.docx  
|-- src/  
|   |-- best_model/  
|   |   |-- best_model.pth  
|   |-- build_test/  
|   |   |-- Dataset_build_voc.py  
|   |   |-- Mask2_GAN.py  
|   |   |-- train.py  
|   |   |-- test.py  
|-- test/  
|   |-- 1.png  
|   |-- 2.png  
|   |-- 3.png  
|   |-- 4.png  
|   |-- 5.png  
|   |-- 6.png  
|-- VOC/
```

3.1 Example test images

Six example dispersion-spectrum images are provided in the test directory:

```
1.png  
2.png  
3.png  
4.png  
5.png  
6.png
```

These images are intended to allow users and reviewers to verify the pretrained-model inference workflow without retraining the network.

The inference script accepts only the file name of an image located in the repository test directory. For example, --image 3.png instructs the program to read test/3.png.

3.2 Pretrained model

The pretrained model parameters are stored in:

```
src/best_model/best_model.pth
```

The inference script locates this checkpoint automatically. Therefore, users do not need to edit the model path when the repository structure is kept unchanged.

3.3 Sample training dataset

The VOC directory is the sample dataset used to demonstrate the training/data-loading workflow. Dataset construction and loading are implemented in:

```
src/build_test/Dataset_build_voc.py
```

Before using a new dataset for training, its directory structure and annotations should be organized consistently with the implementation in Dataset_build_voc.py.

4 Running the inference code

4.1 Clone the repository

First, clone the repository and enter the project directory:

```
git clone https://github.com/shengdate/DIMFormer-GAN.git  
cd DIMFormer-GAN
```

If the environment has not yet been configured, install the required packages as described in Section 2 or use the repository requirements.txt file.

4.2 Run the default example

The simplest command is:

```
python src/build_test/test.py
```

When no image name is provided, the program uses test/2.png as the default input.

4.3 Select another example image

To explicitly select an image, use the --image argument:

```
python src/build_test/test.py --image 2.png
```

Other examples can be tested in the same way:

```
python src/build_test/test.py --image 1.png
python src/build_test/test.py --image 3.png
python src/build_test/test.py --image 4.png
python src/build_test/test.py --image 5.png
python src/build_test/test.py --image 6.png
```

The script does not require an absolute file path. Only the file name from the repository test directory should be supplied.

4.4 Run in a headless environment

By default, the prediction visualization is displayed with Matplotlib after inference. On a remote server or other headless system, use:

```
python src/build_test/test.py --image 2.png --no-show
```

The result is still saved to disk even when the display window is disabled.

4.5 Specify a different output directory

The default output directory is results/. A different directory can be selected with --output-dir:

```
python src/build_test/test.py --image 2.png --output-dir results/example_2
```

5 Inference workflow and output

The released inference script performs the following operations:

3. Reads the selected RGB dispersion-spectrum image.
4. Resizes the input image to 512 x 512 pixels for network inference.
5. Applies the normalization used by the released model.
6. Constructs the DIMFormer-GAN generator.
7. Loads src/best_model/best_model.pth.
8. Predicts query classification scores and instance masks.
9. Retains masks satisfying the configured score and mask thresholds.
10. Optionally applies connected-component refinement.
11. Restores the masks to the original image size.
12. Creates a colored instance overlay and saves the visualization.

For the command:

```
python src/build_test/test.py --image 2.png
```

the default output is:

```
results/2_instance_overlay.png
```

During execution, the terminal reports the project root, checkpoint path, selected test image, output path, computation device, retained query indices, number of detected instances, and final mask-array size.

6 Training the model

The training entry script is:

```
src/build_test/train.py
```

All principal training parameters are centralized in the Config class in train.py. These settings include dataset and input options, train/validation/test split settings, model architecture, Hungarian matching weights, segmentation and adversarial loss weights, batch size, learning rates, number of epochs, random seed, checkpoint saving, and resume options. To reproduce the released training workflow, users can run train.py directly without supplying a separate configuration file. If different training settings are required, edit the corresponding values in train.py before starting the run.

Training with the configured parameters requires only the following command from the repository root:

```
python src/build_test/train.py
```

The dataset-loading implementation is located in:

```
src/build_test/Dataset_build_voc.py
```

and the sample dataset is stored in:

```
VOC/
```

7 Reproducibility information

For the released training and inference workflows, the currently specified software environment is:

- Python 3.9;
- PyTorch 2.0.0+cu118;
- torchvision 0.15.1+cu118;
- NumPy 1.24.3;
- Pillow 11.3.0;
- Matplotlib 3.9.2;
- opencv-python 4.11.0.86;
- SciPy 1.13.1;
- pandas 2.3.3;

For full experimental reproducibility, the public repository should additionally record the operating system, cuDNN version, GPU model and GPU memory, and the principal training hyperparameters.

A reviewer should be able to clone the repository, install the listed dependencies, and execute the following command without changing local file paths:

```
python src/build_test/test.py --image 2.png --no-show
```

The command should load the supplied pretrained checkpoint and save the prediction result in the results directory.

8 Troubleshooting

8.1 Test-image directory not found

The six example images must be located directly under:

```
DIMFormer-GAN/test/
```

They should not be placed under src/test/.

8.2 Checkpoint not found

Confirm that the pretrained model is stored at:

```
DIMFormer-GAN/src/best_model/best_model.pth
```

8.3 Import error

Run the script from the repository root and ensure that all required dependencies have been installed:

```
cd DIMFormer-GAN
python src/build_test/test.py --image 2.png
```

8.4 CUDA error

Verify that the installed PyTorch build is compatible with the local NVIDIA/CUDA environment. The released environment uses the cu118 builds of PyTorch 2.0.0 and torchvision 0.15.1. If CUDA is not available, the inference script automatically falls back to the CPU.

8.5 Running without a display

Use the following command when running on a remote server or other environment without a graphical display:

```
python src/build_test/test.py --image 2.png --no-show
```

9 Citation and contact

The repository accompanies the manuscript:

Intelligent Extraction of Multimodal Surface-Wave Dispersion Curves via Semantic-Constrained Adversarial Instance Segmentation

Citation information should be updated after publication.

For questions regarding DIMFormer-GAN, please contact the corresponding author:

Hongyan Shen